

LINK GITHUB:

<https://github.com/Leo-gsilva/Omi-app>

Arquivo: `./Omi/ViewModel/LivroDeReceitasViewModel.swift`

```
//  
// LivroDeReceitasViewModel.swift  
// Omi  
//  
// Created by Igor Carrasco on 19/08/26.  
//  
  
import Observation  
import CoreData  
  
@Observable  
final class LivroReceitasViewModel {  
    var paginaAtual: Int = 0 // índice da RECEITA  
    dentro da categoria  
    var categoriaAtual: CategoriaReceita = .sobremesa // setada pelas tags  
    private(set) var receitas: [ReceitaModel] = []  
    var livroAberto: Bool = false  
  
    private let repo: ReceitaRepositorio  
    private var observer: NSObjectProtocol?  
    // Recebe protocolo e não a classe  
  
    init(repo: ReceitaRepositorio) {  
        self.repo = repo  
        carregarReceitas()  
        observarMudancas()  
    }  
  
    var receitasFiltradas: [ReceitaModel] {  
        receitas.filter { $0.categoria == categoriaAtual }  
    }  
  
    var totalPaginas: Int {  
        receitasFiltradas.count  
    }  
  
    // Receita correspondete a página atual  
    // Usar para alimentar o tableView  
    var receitaAtual: ReceitaModel? {  
        guard paginaAtual >= 1, paginaAtual <= receitasFiltradas.count else  
        { return nil }  
        return receitasFiltradas[paginaAtual - 1]  
    }  
  
    func carregarReceitas() {  
        do {
```

```
        receitas = try repo.buscarReceitas()
    } catch {
        print("Erro ao buscar receitas: \(error)")
        receitas = []
    }
}

func deletar(_ receita: ReceitaModel) {
    do {
        try repo.deletarReceita(id: receita.id)
        carregarReceitas()
    } catch {
        print("Erro ao deletar receita \(error)")
    }
}

func abrirLivro() {
    guard !livroAberto else { return }
    livroAberto = true
}

func selecionarCategoria(_ categoria: CategoriaReceita) {
    categoriaAtual = categoria
    paginaAtual = 0
}

// func avancar() {
//     guard paginaAtual < totalPaginas - 1 else { return }
//     paginaAtual += 1
// }
//
// func voltar() {
//     guard paginaAtual > 0 else { return }
//     paginaAtual -= 1
// }

private func observarMudancas() {
    observer = NotificationCenter.default.addObserver(
        forName: .NSManagedObjectContextDidSave,
        object: nil,
        queue: .main ){ [weak self] _ in
            self?.carregarReceitas()
        }
}

// Funcs do antigo TelaInicialViewModel
// func abrirLivro() {
//     guard !livroAberto else { return }
//
//     livroAberto = true
// }

// Agora avança dentro da categoria atual
func avancar() {
```

```
    if totalPaginas == 0 {
        let categoriaAntes = categoriaAtual
        proximaCategoria()
        if categoriaAtual != categoriaAntes {
            paginaAtual = totalPaginas > 0 ? 1 : 0
        }

        return
    }
    if paginaAtual < totalPaginas {
        paginaAtual += 1
    } else {
        let categoriaAntes = categoriaAtual
        proximaCategoria()
        if categoriaAtual != categoriaAntes {
            paginaAtual = 0
        }
    }
}

func voltar() {
    if totalPaginas == 0 {
        let categoriaAntes = categoriaAtual
        categoriaAnterior()
        if categoriaAtual != categoriaAntes {
            paginaAtual = totalPaginas > 0 ? totalPaginas : 0
        }

        return
    }
    if paginaAtual > 1 {
        paginaAtual -= 1
    } else {
        let categoriaAntes = categoriaAtual
        categoriaAnterior()
        if categoriaAtual != categoriaAntes {
            paginaAtual = max(totalPaginas, 0)
        }
    }
}

// func tratarPaginaSentinela(_ pagina: Int) {
//     guard totalPaginas > 0 else { return }
//     if pagina == 0 {
//         let categoriaAntes = categoriaAtual
//         categoriaAnterior()
//         paginaAtual = categoriaAtual != categoriaAntes ?
max(totalPaginas, 1) : 1
//     } else if pagina == totalPaginas + 1 {
//         let categoriaAntes = categoriaAtual
//         proximaCategoria()
//         paginaAtual = categoriaAtual != categoriaAntes ?
(totalPaginas > 0 ? 1 : 0) : totalPaginas
//     }
}
```

```
// }

func proximaCategoria() {
    guard let indiceAtual = CategoriaReceita.allCases.firstIndex(of:
categoriaAtual)
    else { return }

    let proximoIndice = min(indiceAtual + 1,
CategoriaReceita.allCases.count - 1)

    categoriaAtual = CategoriaReceita.allCases[proximoIndice]
}

func categoriaAnterior() {
    guard let indiceAtual = CategoriaReceita.allCases.firstIndex(of:
categoriaAtual)
    else { return }

    let indiceAnterior = max(indiceAtual - 1, 0)

    categoriaAtual = CategoriaReceita.allCases[indiceAnterior]
}

deinit {
    if let observer {
        NotificationCenter.default.removeObserver(observer)
    }
}
}
```

Arquivo: ./Omi/ViewModel/DetalhesReceitaViewModel.swift

```
//
// DetalhesReceitaViewModel.swift
// Omi
//
// Created by Leonardo Gonçalves da Silva on 19/08/26.
//
import SwiftUI
import Observation

// pelo que li, é mais tranquilo implementar o coredata usando o
ObservedObject
@Observable
class DetalhesReceitaViewModel {
    var receita: ReceitaModel?

    private let repo: ReceitaRepositorio
```

```
init(receita: ReceitaModel, repo: ReceitaRepositorio) {
    self.receita = receita
    self.repo = repo
    carregarDetalhes()
}

func carregarDetalhes() {
    guard let id = receita?.id else { return }
    do {
        receita = try repo.buscarReceita(id: id)
    } catch {
        print("Erro ao carregar detalhes: \(error)")
    }
}
}
```

Arquivo: `./Omi/ViewModel/CriarReceitaViewModel.swift`

```
//
// CriarReceitaViewModel.swift
// Omi
//
// Created by Leonardo Gonçalves da Silva on 18/08/26.
//
import Observation
import Foundation

@Observable
final class CriarReceitaViewModel {
    private let repo: ReceitaRepositorio
    // var receita: ReceitaModel?

    init(repo: ReceitaRepositorio) {
        self.repo = repo

        // if modo {
        //     preencherComReceitaExistente(receita)
        // }

    }

    var modo: Bool = false

    // Receita
    var titulo = ""
    var categoria: CategoriaReceita = .refeicao
    var descricao = ""
    var tempoDePreparoTexto = ""
    var porcoesTexto = ""
    var dificuldade = ""
}
```

```
var imagem: Data?

// Ingredientes
// Lista dos itens que o usuário foi adicionando na tela
var ingredientesAdicionados: [IngredienteAdicionado] = []

// Campos temporários para digitar
var nomeIngredienteTexto = ""
var quantidadeTexto = ""
var medidaTexto = ""

// Passos
var passosAdicionados: [PassoAdicionado] = []

var nomeDoPasso: String = ""
var descricaoDoPasso: String = ""
var tempoPassoTexto: String = ""
var imagemPasso: Data?

// FUNÇÕES
// INGREDIENTE
func adicionarIngrediente() {
    guard !nomeIngredienteTexto.isEmpty,
          let quantidade = Double(quantidadeTexto),
          !medidaTexto.isEmpty else { return }

    let novoItem = IngredienteAdicionado(nome: nomeIngredienteTexto,
quantidade: quantidade, medida: medidaTexto)
    ingredientesAdicionados.append(novoItem)

    // Limpa os campos para o próximo ingrediente
    nomeIngredienteTexto = ""
    quantidadeTexto = ""
    medidaTexto = ""
}

// ADICIONAR PASSO
func adicionarPasso() {
    guard !nomeDoPasso.isEmpty, !descricaoDoPasso.isEmpty else { return
}

    let passo = PassoAdicionado(etapa: passosAdicionados.count + 1,
nome: nomeDoPasso, texto: descricaoDoPasso, tempoEstimado:
Int(tempoPassoTexto) ?? 0)

    passosAdicionados.append(passo)

    nomeDoPasso = ""
    descricaoDoPasso = ""
    tempoPassoTexto = ""
}

// MARK: - Actions
func preencherComReceitaExistente(_ receita: ReceitaModel) {
```

```

        titulo = receita.titulo
        categoria = receita.categoria
        descricao = receita.descricao
        tempoDePreparoTexto = String(receita.tempoDePreparo)
        porcoesTexto = receita.porcoes
        dificuldade = receita.dificuldade ?? ""

        ingredientesAdicionados = receita.ingredientes.compactMap {
            guard let qtd = Double($0.quantidade) else { return nil }
            return IngredienteAdicionado(nome: $0.nome, quantidade: qtd,
medida: $0.medida)
        }

        passosAdicionados = receita.passos.map {
            PassoAdicionado(etapa: Int($0.etapa), nome: $0.nome, texto:
$0.texto, tempoEstimado: Int($0.tempoEstimado))
        }
    }

    func salvarReceitaNoBanco() {
        do {
            try repo.criarReceita(
                titulo: titulo,
                categoria: categoria.rawValue,
                descricao: descricao,
                imagem: imagem,
                tempoDePreparo: Int16(tempoDePreparoTexto) ?? 0,
                porcoes: porcoesTexto,
                dificuldade: dificuldade,
                ingredientes: ingredientesAdicionados,
                passos: passosAdicionados
            )
            print("Receita e ingredientes salvos perfeitamente!")
        } catch {
            print("Erro ao salvar: \(error)")
        }
    }
}

```

Arquivo: ./Omi/ViewModel/CategoriaViewModel.swift

```

//
//  CategoriaViewModel.swift
//  Omi
//
//  Created by Igor Carrasco on 19/08/26.
//

import Observation

```

```
@Observable
final class CategoriaViewModel {
    var categoriaAtual: CategoriaReceita = .cafeDaManha

    func proximaCategoria() {
        guard let indiceAtual = CategoriaReceita.allCases.firstIndex(of:
categoriaAtual) else { return }

        let proximoIndice = min(indiceAtual + 1,
CategoriaReceita.allCases.count - 1)

        categoriaAtual = CategoriaReceita.allCases[proximoIndice]
    }

    func categoriaAnterior() {
        guard let indiceAtual = CategoriaReceita.allCases.firstIndex(of:
categoriaAtual) else { return }

        let indiceAnterior = max(indiceAtual - 1, 0)

        categoriaAtual = CategoriaReceita.allCases[indiceAnterior]
    }
}
```

Arquivo: ./Omi/ViewModel/IngredientesViewModel.swift

```
//
//  IngredientesViewModel.swift
//  Omi
//
//  Created by Leonardo Gonçalves da Silva on 17/08/26.
//
import SwiftUI
import Observation

@Observable
class IngredientesViewModel {
    // Look how clean this is! No @Published needed.
    var itens: [IngredienteCadastradoModel] = []

    // We make the repo private so the View can't touch it directly.
    private let repo: ReceitaRepositorio

    init(repo: ReceitaRepositorio) {
        self.repo = repo
        carregarIngredientes()
    }
}
```

```
func carregarIngredientes() {
    do {
        itens = try repo.buscarIngredientes()
    } catch {
        print("Erro ao buscar: \(error)")
    }
}

func adicionar(nome: String) {
    guard !nome.isEmpty else { return }
    do {
        _ = try repo.criarIngredienteAvulso(nome: nome)
        carregarIngredientes()
    } catch {
        print("Erro ao adicionar: \(error)")
    }
}

func deletar(offsets: IndexSet) {
    offsets.map { itens[$0] }.forEach { ingrediente in
        do {
            try repo.deletarIngrediente(id: ingrediente.id)
        } catch {
            print("Erro ao deletar: \(error)")
        }
    }
    carregarIngredientes()
}
}
```

Arquivo: ./Omi/ViewModel/OnboardingViewModel.swift

```
//
// OnboardingViewModel.swift
// Omi
//
// Created by Guilherme Alves de Souza on 16/08/26.
//

import SwiftUI
import Observation

@Observable
@MainActor
final class OnboardingViewModel {
    var postitAtivo: Int? = nil
    var paginaAtual: Int = 0
    var finalizado: Bool = false
}
```

```
let totalDePaginas = 6

let paginas: [OnboardingModel] = [
    OnboardingModel(
        titulo: "O que o app faz?",
        descricao: "Esse é o seu ",
        palavraDestaque: "álbum \nde receitas.",
        palavraDestaque2: nil,
        imagem: "AlbumOnboarding"
    ),

    OnboardingModel(
        titulo: "Crie categorias",
        descricao: "Organize suas receitas,\n por ",
        palavraDestaque: "categorias ",
        palavraDestaque2: "etiquetas.",
        imagem: "Albumcortadoonboarding"
    ),

    OnboardingModel(
        titulo: "Adicione Receitas",
        descricao: "Você pode ",
        palavraDestaque: "adicionar novas receitas",
        palavraDestaque2: " para seu álbum.",
        imagem: "ImageAdicionarpag3"
    ),

    OnboardingModel(
        titulo: "Edite suas receitas",
        descricao: "No botão Editar, você pode  no álbum.  ",
        palavraDestaque: "mudar suas receitas",
        palavraDestaque2: nil,
        imagem: "Editarpag4",
    ),

    OnboardingModel(
        titulo: "Veja sua receitas ",
        descricao: "na sua\n receita, você pode\n",
        palavraDestaque: "tocar ",
        palavraDestaque2: "vê-la com\n mais detalhes.",
        imagem: "livropag5"
    )
]

// var totalDePaginas: Int {
//     paginas.count
// }

// var paginaAtualModel: OnboardingModel {
//     paginas[paginaAtual+1]
// }

func continuar() {
```

```
        if paginaAtual < totalDePaginas - 1 {
            paginaAtual += 1
            print(paginaAtual)
        } else {
            finalizado = true
        }
    }

    func voltar() {
        guard paginaAtual > 0 else { return }

        paginaAtual -= 1
    }

    func iniciarAnimacaoPostIts() async {

        while !Task.isCancelled {

            for index in 0..<5 {

                withAnimation(.easeInOut(duration: 0.35)) {
                    postitAtivo = index
                }

                try? await Task.sleep(for: .milliseconds(500))

                withAnimation(.easeInOut(duration: 0.35)) {
                    postitAtivo = nil
                }

                try? await Task.sleep(for: .milliseconds(150))
            }
        }
    }

    // private func finalizarOnboarding() {
    //     finalizado = true
    //     print("Onboarding finalizado")
    // }
    }
```

Arquivo: ./Omi/Navegacao/Rota.swift

```
//
// NavigationModel.swift
// Omi
//
// Created by Igor Carrasco on 20/08/26.
//
```

```
import Foundation

enum Rota: Hashable {
    case telaInicial
    case detalheReceita(ReceitaModel)
    case criarReceita
    // case listaIngredientes
    case onboarding
    //case categoriaSheetView
}

extension Rota: Identifiable {
    // Como Rota já é hashable, ela serve como seu próprio id
    var id: Self { self }
}
```

Arquivo: `./Omi/Navegacao/AppRouter.swift`

```
//
// AppRouter.swift
// Omi
//
// Created by Igor Carrasco on 20/08/26.
//

import Observation
import SwiftUI

@Observable
final class AppRouter {
    var path = NavigationPath()

    var sheetAtual: Rota?

    func push(_ rota: Rota) {
        path.append(rota)
    }

    func pop() {
        guard !path.isEmpty else { return }
        path.removeLast()
    }

    func popATodas() {
        path.removeLast(path.count)
    }

    func apresentarSheet(_ rota: Rota) {
```

```
        sheetAtual = rota
    }

    func fecharSheet() {
        sheetAtual = nil
    }

    // não é push ou pop, ele troca a raiz do app inteiro
    func finalizarOnboarding() {
        UserDefaults.standard.set(true, forKey: "onboardingConcluido")
    }
}
```

Arquivo: `./Omi/Model/DataModel.swift`

```
//
// ReceitaModel.swift
// Omi
//
// Created by Igor Carrasco on 19/08/26.
//

import Foundation

// Agora a struct temporária guarda apenas a String do nome!
struct IngredienteAdicionado: Identifiable {
    let id = UUID()
    let nome: String
    let quantidade: Double
    let medida: String
}

struct PassoAdicionado: Identifiable {
    let id = UUID()
    let etapa: Int
    let nome: String
    let texto: String
    let tempoEstimado: Int
    // let imagem: Data?
}

// Versão de leitura de uma receita, só para exibição
// Nenhuma View deve conhecer a entity `Receita` Core Data
struct ReceitaModel: Identifiable, Hashable {
    let id: UUID
    var titulo: String
    var categoria: CategoriaReceita
    var descricao: String
    var imagem: Data?
```

```
var tempoDePreparo: Int16
var porcoes: String
var dificuldade: String?
var dataCriacao: Date
var dataAtualizacao: Date?
var ingredientes: [IngredienteModel]
var passos: [PassoModel]
}

struct IngredienteModel: Identifiable, Hashable {
    let id: UUID
    var nome: String
    var quantidade: String
    var medida: String
}

struct PassoModel: Identifiable, Hashable {
    let id: UUID
    var etapa: Int16
    var nome: String
    var texto: String
    var tempoEstimado: Int16
    // var imagem: Data?
}

// Representa a entidade Ingrediente pura
struct IngredienteCadastradoModel: Identifiable, Hashable {
    let id: UUID
    var nome: String
}

extension Ingrediente {
    func toModel() -> IngredienteCadastradoModel {
        IngredienteCadastradoModel(id: id ?? UUID(), nome: nome ?? "")
    }
}

extension PassoReceita {
    func toModel() -> PassoModel {
        PassoModel(id: id ?? UUID(), etapa: etapa, nome: nome ?? "", texto:
texto ?? "", tempoEstimado: tempoEstimado)
    }
}

// MARK: - Mapeamento Entidade ->
// Único lugar do app que sabe transformar NSManagedObject em Struct de
comínio
extension Receita {
    func toModel() -> ReceitaModel {
        let ingredientesModel: [IngredienteModel] = (ingredientesDaReceita
as? Set<IngredienteDaReceita> ?? [])
            .compactMap { relacao in
                guard let ingrediente = relacao.ingrediente else { return
nil }
            }
    }
}
```

```
        return IngredienteModel(
            id: relacao.id ?? UUID(),
            nome: ingrediente.nome ?? "",
            quantidade: relacao.quantidade ?? "",
            medida: relacao.medida ?? ""
        )
    }

    let passosModel: [PassoModel] = (passo as? Set<PassoReceita> ?? [])
        .map { passo in
            PassoModel(
                id: passo.id ?? UUID(),
                etapa: passo.etapa + 1,
                nome: passo.nome ?? "",
                texto: passo.texto ?? "",
                tempoEstimado: passo.tempoEstimado
                //imagem: passo.imagem
            )
        }
        .sorted { $0.etapa < $1.etapa }

    return ReceitaModel(
        id: id ?? UUID(),
        titulo: titulo ?? "",
        categoria: CategoriaReceita(
            rawValue: categoria ?? "") ?? .sobremesa,
        descricao: descricao ?? "",
        imagem: imagem,
        tempoDePreparo: tempoDePreparo,
        porcoes: porcoes ?? "",
        dataCriacao: dataCriacao ?? Date(),
        dataAtualizacao: dataAtualizacao,
        ingredientes: ingredientesModel,
        passos: passosModel)
    }
}
```

Arquivo: ./Omi/Model/CategoriaModel.swift

```
//
//  CategoriaModel.swift
//  Omi
//
//  Created by Igor Carrasco on 19/08/26.
//

import Foundation
import SwiftUI
```

```
enum CategoriaReceita: String, CaseIterable, Identifiable {
    case cafeDaManha = "Café da Manhã"
    case refeicao = "Refeição"
    case saudavel = "Saudável"
    case sobremesa = "Sobremesa"
    case lanche = "Lanche"

    var id: String {
        rawValue
    }

    // Ajuste os RGB pra bater com os hex exatos do Figma se tiver acesso a eles
    var cor: Color {
        switch self {
            case .sobremesa: return Color(.corSobremesa)
            case .refeicao: return Color(.corRefeicao)
            case .saudavel: return Color(.corSaudavel)
            case .lanche: return Color(.corLanche)
            case .cafeDaManha: return Color(.corCafe)
        }
    }
}
```

Arquivo: ./Omi/Model/DetalheModel.swift

```
//
// DetalheModel.swift
// Omi
//
// Created by Leonardo Gonçalves da Silva on 21/08/26.
//
enum TipoDetalhe {
    case tempo(Int16)
    case pessoas(String)

    var textoFormatado: String {
        switch self {
            case .tempo(let minutos):
                return "\(minutos) min"
            case .pessoas(let quantidade):
                return "\(quantidade) pessoas"
        }
    }

    var iconePadrao: String {
        switch self {
            case .tempo:
```

```
        return "clock"
    case .pessoas:
        return "chart.pie"
    }
}
```

Arquivo: `./Omi/Model/OnboardingModel.swift`

```
//
//  OnboardingModel.swift
//  Omi
//
//  Created by Leonardo Gonçalves da Silva on 17/08/26.
//
import Foundation

struct OnboardingModel {
    let titulo: String
    let descricao: String
    let palavraDestaque: String
    let palavraDestaque2: String?
    let imagem: String
}
```

Arquivo: `./Omi/View/ReceitaPageView.swift`

```
//
//  ReceitaPageView.swift
//  Omi
//
//  Created by Igor Carrasco on 19/08/26.
//

import SwiftUI

struct ReceitaPageView: View {
    //ReceitaModel, não Receita (entidade) – Não chama CoreData
    let receita: ReceitaModel

    @Environment(AppRouter.self) private var router

    var body: some View {
        ScrollView{
```

```
VStack(alignment: .leading, spacing: 12) {
    Section{
        Text(receita.titulo)
            .font(.title)
            .bold()

        Text("data de criação: \
(receita.dataCriacao.formatted(.dateTime))")
            .font(.caption)

        Text(receita.categoria.rawValue)
            .font(.caption)
            .foregroundColor(.secondary)
        HStack{
            Text("Tempo: \((receita.tempoDePreparo) min")
            Text("Porções: \((receita.porcoes)")
        }
        Text("Categoria: \((receita.categoria.rawValue)")
    }

    Divider()

    Text(receita.descricao)

    if !receita.passos.isEmpty {
        Text("Modo de preparo")
            .font(.headline)

        ForEach(receita.passos) { passo in
            Text("\((passo.etapa). \((passo.nome)")
                .fontWeight(.semibold)
            Text(passo.texto)
        }
    }

    Button{
        router.push(.detalheReceita(receita))
    } label: {
        Label("Ver mais detalhes", systemImage: "arrow.up.right")
    }
        .padding(.top)
}
.contentShape(Rectangle())
.padding()
.onTapGesture {
    router.push(.detalheReceita(receita))
}
}

}

#Preview {
    NavigationStack{
        ReceitaPageView(
```

```

        receita: ReceitaModel(
            id: UUID(),
            titulo: "Bolo de Cenoura",
            categoria: .sobremesa,
            descricao: "Receita de teste para Preview.",
            imagem: nil,
            tempoDePreparo: 45,
            porcoes: "8",
            dificuldade: "Fácil",
            dataCriacao: Date(),
            dataAtualizacao: nil,
            ingredientes: [
                IngredienteModel(id: UUID(), nome: "Cenoura",
quantidade: "3", medida: "unidades"),
                IngredienteModel(id: UUID(), nome: "Farinha",
quantidade: "2", medida: "xícaras")
            ],
            passos: [
                PassoModel(id: UUID(), etapa: 1, nome: "Misture",
texto: "Bata tudo no liquidificador.", tempoEstimado: 5),
                PassoModel(id: UUID(), etapa: 2, nome: "Asse", texto:
"Leve ao forno por 40 min.", tempoEstimado: 40)
            ]
        )
    )
}
.environment(AppRouter())
}

```

Arquivo: ./Omi/View/Componentes/BarradeTags.swift

```

//
//  BarradeTags.swift
//  Omi
//
//  Created by Guilherme Alves de Souza on 21/08/26.
//

import SwiftUI

struct BarraDeTags: View {
    @Bindable var viewModel: LivroReceitasViewModel

    private let ordemCategorias: [CategoriaReceita] = [
        .cafeDaManha, .refeicao, .saudavel, .sobremesa, .lanche
    ]

    private let tagPorCategoria: [CategoriaReceita: String] = [
        .cafeDaManha: "TagVermelha",

```

```
.refeicao:      "TagAzul",
.saudavel:     "TagVerde",
.sobremesa:    "TagAmarela",
.lanche:       "TagLaranja"
]

var body: some View {
    GeometryReader { geo in

        HStack(spacing: 3) {

            ForEach(ordemCategorias) { categoria in

                Image(tagPorCategoria[categoria] ?? "TagVermelha")
                    .resizable()
                    .scaledToFit()
                    .frame(width: geo.size.width * 0.10)
                    .offset(
                        y: viewModel.categoriaAtual == categoria
                        ? -14
                        : 0
                    )
                    .contentShape(
                        Rectangle()
                    )
                    .onTapGesture {
                        viewModel.selecionarCategoria(categoria)
                    }
                    .animation(
                        .spring(
                            response: 0.35,
                            dampingFraction: 0.7
                        ),
                        value: viewModel.categoriaAtual
                    )
            }
        }
        .padding(.horizontal, 150)
    }
}

#Preview {
    BarraDeTags(viewModel: .preview)
}
```

Arquivo: ./Omi/View/Componentes/postitOnboarding.swift

```
//  
// postitOnboarding.swift  
// On bording  
//  
// Created by Guilherme Alves de Souza on 17/08/26.  
//  
  
import SwiftUI  
struct PostitOnboarding: View {  
  
    let imagem: String  
    let ativo: Bool  
  
    var body: some View {  
        Image(imagem)  
            .resizable()  
            .scaledToFit()  
            .offset(y: ativo ? -20 : 0)  
            .animation(  
                .easeInOut(duration: 0.35),  
                value: ativo  
            )  
    }  
}
```

Arquivo: ./Omi/View/Componentes/CartaoClaro.swift

```
//  
// CartaoClaro.swift  
// Omi  
//  
// Created by Leonardo Gonçalves da Silva on 20/08/26.  
//  
import SwiftUI  
  
struct CartaoClaro<Content: View>: View {  
    @ViewBuilder var conteudo: Content  
  
    var body: some View {  
        VStack(alignment: .leading, spacing: 12) {  
            conteudo  
        }  
        .padding(16)  
        .frame(maxWidth: .infinity, alignment: .leading)  
        .background(Color(.corDoFundoTexto))  
        .clipShape(RoundedRectangle(cornerRadius: 16))  
    }  
}
```

```
}
#Preview {
  CartaoClaro {
    VStack(alignment: .leading, spacing: 10) {
      Text("Etapa 2: OLA")
        .font(FontesApp.subtitulo)
        .foregroundColor(.cordosTextos)

      Divider()

      Text("oi")
        .font(FontesApp.corpo)
        .foregroundColor(.cordosTextos)
    }
  }
}
```

Arquivo: `./Omi/View/Componentes/EtiquetaSeta.swift`

```
////
////  EtiquetaSeta.swift
////  Omi
////
////  Created by Leonardo Gonçalves da Silva on 23/08/26.
////
//import SwiftUI
//
//struct EtiquetaSeta: Shape {
//    func path(in rect: CGRect) -> Path {
//        let ponta = rect.height / 2
//        var path = Path()
//        path.move(to: CGPoint(x: 0, y: 0))
//        path.addLine(to: CGPoint(x: rect.width - ponta, y: 0))
//        path.addLine(to: CGPoint(x: rect.width, y: rect.height / 2))
//        path.addLine(to: CGPoint(x: rect.width - ponta, y: rect.height))
//        path.addLine(to: CGPoint(x: 0, y: rect.height))
//        path.closeSubpath()
//        return path
//    }
//}
```

Arquivo: `./Omi/View/Componentes/ReceitaHeroView.swift`

```
//
//  ReceitaHeroView.swift
```

```
// Omi
//
// Created by Leonardo Gonçalves da Silva on 20/08/26.
//
import SwiftUI

struct ReceitaHeroView: View {
    let imagemData: Data?
    let titulo: String

    var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            GeometryReader { geo in
                Group {
                    if let imagemData, let uiImage = UIImage(data:
imagemData) {
                        Image(uiImage: uiImage)
                            .resizable()
                            .scaledToFill()
                    } else {
                        Image("Bolo")
                            .resizable()
                            .scaledToFill()
                    }
                }
                .frame(width: geo.size.width, height: geo.size.height)
                .clipped()
            }
            .aspectRatio(308.0 / 188.0, contentMode: .fit)
            .clipShape(RoundedRectangle(cornerRadius: 18))

            Text(titulo)
                .font(FontesApp.titulo)
                .foregroundColor(.cordosTextos)
                .padding(.horizontal, 16)
                .padding(.vertical, 10)
                .frame(maxWidth: .infinity, alignment: .leading)
                .background(Color(.cordoFundoTexto))
                .clipShape(RoundedRectangle(cornerRadius: 11))
        }
    }
}

#Preview {
    ReceitaHeroView(imagemData: nil, titulo: "olha")
}
```

Arquivo: ./Omi/View/Componentes/TituloSecao.swift

```
//  
// TituloSecao.swift  
// Omi  
//  
// Created by Leonardo Gonçalves da Silva on 20/08/26.  
//  
import SwiftUI  
  
struct TituloSecao: View {  
    let texto: String  
  
    var body: some View {  
        Text(texto)  
            .font(FontesApp.subtitulo)  
            .foregroundColor(.cordosTextos)  
    }  
}  
#Preview {  
    TituloSecao(texto: "Título da seção")  
}
```

Arquivo: `./Omi/View/Componentes/CategoriaSheetView.swift`

```
////  
//// CategoriaSheetView.swift  
//// Omi  
////  
//// Created by Leonardo Gonçalves da Silva on 23/08/26.  
////  
//  
//import SwiftUI  
//  
//struct CategoriaSheetView: View {  
//    @Environment(\.dismiss) private var fechar  
//  
//    let categoriaSelecionada: CategoriaReceita?  
//    var aoSelecionar: (CategoriaReceita) -> Void  
//  
//    var body: some View {  
//        ZStack {  
//            Color(.cordoFundo)  
//                .ignoresSafeArea()  
//  
//            VStack(alignment: .leading, spacing: 20) {  
//                HStack {  
//                    Button(action: { fechar() }) {  
//                        Image(systemName: "xmark")  
//                            .font(.system(size: 16, weight: .semibold))  
//                    }  
//                }  
//            }  
//        }  
//    }  
//}
```

```

//                                .foregroundColor(.cordosTextos)
//                                .padding(10)
//                                .background(.ultraThinMaterial, in: Circle())
//                            }
//
//                            Spacer()
//
//                            Text("Adicionar categoria")
//                                .font(FontesApp.subtitulo)
//                                .foregroundColor(.cordosTextos)
//
//                            Spacer()
//
//                            Button(action: { fechar() }) {
//                                Image(systemName: "checkmark")
//                                    .font(.system(size: 16, weight: .semibold))
//                                    .foregroundColor(.white)
//                                    .padding(10)
//                                    .background(Color(.cordosTextos), in:
Circle())
//                            }
//                        }
//
//                        Text("Selecione uma etiqueta com o nome da categoria que
você quer escolher para sua receita.")
//                            .font(FontesApp.corpo)
//                            .foregroundColor(.cordosTextos.opacity(0.7))
//
//                        VStack(spacing: 16) {
//                            ForEach(CategoriaReceita.allCases) { categoria in
//                                CategoriaEtiquetaButton(
//                                    categoria: categoria,
//                                    selecionada: categoria ==
categoriaSelecionada,
//                                    aoSelecionar: {
//                                        aoSelecionar(categoria)
//                                        fechar()
//                                    }
//                                )
//                            }
//                        }
//
//                        Spacer()
//                    }
//                .padding(20)
//            }
//        }
//    }
//}
//
//#Preview {
//    CategoriaSheetView(categoriaSelecionada: .sobremesa, aoSelecionar: {
_ in })

```

```
//}
```

Arquivo: `./Omi/View/Componentes/CapsulaDetalhes.swift`

```
//  
//  Capsula.swift  
//  Omi  
//  
//  Created by Leonardo Gonçalves da Silva on 21/08/26.  
//  
import SwiftUI  
  
struct CapsulaDetalhes: View {  
    let detalhe: TipoDetalhe  
  
    var body: some View {  
        HStack(spacing: 8) {  
  
            Image(systemName: detalhe.iconePadrao)  
                .font(FontesApp.Semibold)  
                .foregroundColor(.cordosTextos)  
  
            Text(detalhe.textoFormatado)  
                .font(FontesApp.Semibold)  
                .foregroundColor(.cordosTextos)  
  
        }  
        .padding(.vertical, 8)  
        .padding(.horizontal)  
        .background(.corFundoCapsula)  
  
        .clipShape(Capsule())  
        .shadow(radius: 2)  
    }  
}  
  
#Preview {  
    CapsulaDetalhes(detalhe: .tempo(100))  
}
```

Arquivo: `./Omi/View/Componentes/TrocarPagina.swift`

```
//  
//  TrocarPagina.swift  
//  Omi
```

```
//  
// Created by Guilherme Alves de Souza on 19/08/26.  
//  
  
import SwiftUI  
  
struct TrocarPagina: View {  
    @Bindable var viewModel: LivroReceitasViewModel  
  
    let voltar: () -> Void  
    let avancar: () -> Void  
  
    var body: some View {  
  
        GeometryReader { geo in  
  
            HStack {  
  
                Button(action: voltar) {  
                    Image(systemName: "arrow.left")  
                        .foregroundColor(.black)  
                        .font(.system(size: 25))  
                        .frame(  
                            maxWidth: .infinity,  
                            maxHeight: .infinity  
                        )  
                }  
                .frame(  
                    width: geo.size.width * 0.30,  
                    height: 50  
                )  
                .contentShape(Rectangle())  
                .glassEffect()  
  
                Text("\((viewModel.livroAberto ? viewModel.paginaAtual :  
0))/\((viewModel.totalPaginas)")  
                    .font(  
                        .system(  
                            size: 20,  
                            weight: .semibold  
                        )  
                    )  
                .frame(  
                    width: geo.size.width * 0.20,  
                    height: 50  
                )  
  
                Button(action: avancar) {  
                    Image(systemName: "arrow.right")  
                        .foregroundColor(.black)  
                        .font(.system(size: 25))  
                        .frame(  
                            maxWidth: .infinity,  
                            maxHeight: .infinity  
                        )  
                }  
            }  
        }  
    }  
}
```

```

        )
    }
    .frame(
        width: geo.size.width * 0.30,
        height: 50
    )
    .contentShape(Rectangle())
    .glassEffect()
}
.frame(maxWidth: .infinity)
}
}
}

#Preview {
    TrocarPagina(
        viewModel: LivroReceitasViewModel.preview,
        voltar: {},
        avancar: {}
    )
}

```

Arquivo: ./Omi/View/Componentes/DividerPersonalizado.swift

```

//
// DividerDoApp.swift
// Omi
//
// Created by Leonardo Gonçalves da Silva on 21/08/26.
//
import SwiftUI

struct DividerPersonalizado: View {
    var body: some View {
        Rectangle()
            .fill(Color(.corDivider))
            .frame(height: 2)
    }
}

#Preview{
    DividerPersonalizado()
}

```

Arquivo: ./Omi/View/Componentes/ProgressoOnboarding.swift

```
//
// progressoOnboarding.swift
// On bording
//
// Created by Guilherme Alves de Souza on 16/08/26.
//

import SwiftUI

struct ProgressoOnboarding: View {

    let paginaAtual: Int
    let totalDePaginas: Int

    var body: some View {
        HStack(spacing: 7) {

            ForEach(0..
```

Arquivo: ./Omi/View/Componentes/BotaoOnboarding.swift

```
//
// BotaoOnboarding.swift
// On bording
//
// Created by Guilherme Alves de Souza on 16/08/26.
//

import SwiftUI

struct BotaoOnboarding: View {
```

```
var textoBotao: String = "Confirmar"
let action: () -> Void

var body: some View {
    Button(action: action) {
        Text(textoBotao)
            .font(FontesApp.Botao)
            .foregroundColor(Color.cordoFundo)
            .frame(maxWidth: .infinity)
            .frame(height: 44)
            .background(Color.cordoBotao)
            .clipShape(
                RoundedRectangle(cornerRadius: 20)
            )
    }
}

#Preview {
    BotaoOnboarding(
        action: {}
    )
}
```

Arquivo: `./Omi/View/Componentes/CategoriaEtiquetaButton.swift`

```
////
//// CategoriaEtiquetaButton.swift
//// Omi
////
//// Created by Leonardo Gonçalves da Silva on 23/08/26.
////
//import SwiftUI
//
//struct CategoriaEtiquetaButton: View {
//    let categoria: CategoriaReceita
//    let selecionada: Bool
//    var aoSelecionar: () -> Void
//
//    var body: some View {
//        Button(action: aoSelecionar) {
//            HStack {
//                Text(categoria.rawValue)
//                    .font(FontesApp.subtitulo)
//                    .foregroundColor(.white)
//
//                Spacer()
//            }
//        }
//    }
//}
```

```

//          if selecionada {
//              Image(systemName: "checkmark")
//                  .foregroundColor(.white)
//                  .padding(.trailing, 24)
//          }
//      }
//      .padding(.leading, 20)
//      .padding(.vertical, 18)
//      .background(selecionada ? categoria.cor :
categoria.cor.opacity(0.40))
//      .clipShape(EtiquetaSeta())
//  }
//  .buttonStyle(.plain)
//  }
//}
//
//#Preview {
//    CategoriaEtiquetaButton(categoria: .cafeDaManha, selecionada: true,
aoSelecionar: {})
//    .padding()
//}

```

Arquivo: [./Omi/View/Componentes/ReceitaEtapaCardView.swift](#)

```

//
//  ReceitaEtapaCardView.swift
//  Omi
//
//  Created by Leonardo Gonçalves da Silva on 20/08/26.
//
import SwiftUI

struct ReceitaEtapaCardView: View {
    let numero: Int16
    let nome: String
    let texto: String
    //    let imagemData: Data?

    var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            CartaoClaro {
                VStack(alignment: .leading, spacing: 10) {
                    Text("Etapa \(numero): \(nome)")
                        .font(FontesApp.subtitulo)
                        .foregroundColor(.cordosTextos)

                    DividerPersonalizado()

                    Text(texto)
                }
            }
        }
    }
}

```

```

                .font(FontesApp.corpo)
                .foregroundColor(.cordosTextos)
            }
        }

//          if let imageData, let uiImage = UIImage(data: imageData) {
//              GeometryReader { geo in
//                  Image(uiImage: uiImage)
//
//                      .resizable()
//                      .scaledToFill()
//                      .frame(width: geo.size.width, height:
geo.size.height)
//                      .clipped()
//              }
//              .aspectRatio(308.0 / 188.0, contentMode: .fit)
//              .clipShape(RoundedRectangle(cornerRadius: 16))
//          }
        }
    }
}
#Preview {
    ReceitaEtapaCardView(numero: 1, nome: "Leo", texto: "ola")
}

```

Arquivo: `./Omi/View/Componentes/LivroInterativo.swift`

```

//
//  LivroInterativo.swift
//  Omi
//
//  Created by Guilherme Alves de Souza on 24/08/26.
//

import SwiftUI

struct LivroInterativo: View {
    @Bindable var viewModel: LivroReceitasViewModel
    // @Bindable var viewModelDetalhes:
    @State private var pesquisa = ""
    // @State private var mostrarLivroAberto = false

    private let animacaoLivro = Animation.spring(
        response: 0.65,
        dampingFraction: 0.78
    )
    private let coresPorCategoria: [CategoriaReceita: String] = [
        .cafeDaManha: "albumAbertoVermelho",

```

```

        .refeicao:      "albumAbertoAzul",
        .saudavel:     "albumAbertoVerde",
        .sobremesa:    "albumAbertoAmarelo",
        .lanche:       "albumAbertoLaranja"
    ]

    private var nomePaginaAtual: String {
        coresPorCategoria[viewModel.categoriaAtual] ??
"albumAbertoVermelho"
    }
    var body: some View {
        GeometryReader { geo in
            ZStack(alignment: .leading) {
                if viewModel.livroAberto {
                    BarraDeTags(viewModel: viewModel)

                    Image(nomePaginaAtual)
                        .resizable()
                        .scaledToFit()
                        .frame(width: geo.size.width * 1.00)
                        .opacity(viewModel.livroAberto ? 1 : 0)
                        .ignoresSafeArea(edges: .all)
                        .id(nomePaginaAtual)
                        .transition(.opacity)
                        .animation(.easeInOut(duration: 0.35), value:
nomePaginaAtual)
                        .allowsHitTesting(false)

                    if viewModel.livroAberto {
                        LivroReceitasViewSimples(viewModel: viewModel)
                            .frame(
                                width: geo.size.width * 0.75,
                                height: geo.size.height * 0.55
                            )
                            .padding(.leading, geo.size.width * 0.12)
                            .transition(.opacity)
                            .zIndex(2)
                    }

                    Image("ReceitaTelaInicial")
                        .resizable()
                        .scaledToFit()
                        .frame(width: geo.size.width * 0.90)
                        .rotation3DEffect(
                            .degrees(viewModel.livroAberto ? -90 : 0),
                            axis: (x: 0, y: 1, z: 0),
                            anchor: .leading,
                            perspective: 0.35
                        )
                        .shadow(color: .black.opacity(0.25), radius: 14, x: 8,
y: 10)
                        .padding(.horizontal, 30)
                }
            }
        }
    }
}

```

```
        }

        .onTapGesture {

            abrirLivro()
        }
        .animation(
            .easeInOut(duration: 0.4),
            value: viewModel.paginaAtual
        )

    }
}
private func abrirLivro() {

    guard !viewModel.livroAberto else {
        return
    }

    withAnimation(animacaoLivro) {
        viewModel.abrirLivro()
    }

    DispatchQueue.main.asyncAfter(
        deadline: .now() + 0.50
    ) {
        withAnimation(.easeInOut(duration: 0.15)) {
            viewModel.livroAberto = true
        }
    }
}

#Preview {
    LivroInterativo(viewModel: LivroReceitasViewModel.preview)
}
```

Arquivo: ./Omi/View/TelaDeApresetacao.swift

```
//
//  TeladeApresentação.swift
//  Omi
//
```

```
// Created by Guilherme Alves de Souza on 18/08/26.
//

import SwiftUI

struct TelaDeApresetacao: View {
    @Bindable var viewModel: OnboardingViewModel

    var body: some View {
        GeometryReader { geo in
            VStack(spacing: 0) {
                Spacer()
                Text("Bem-Vindo(a) ao 0Vô")
                    .font(FontesApp.titulo)
                    .foregroundColor(.black.opacity(0.7))
                    .padding(.top, geo.size.height * 0.035)

                Spacer()

                Text("0 seu app para\n anotar as receitas\n do dia a dia!")
                    .multilineTextAlignment(.center)
                    .font(FontesApp.tituloComTexto)
                    .foregroundColor(Color.cordosTextos)

                Spacer()

                Image("Ovo0")
                    .resizable()
                    .scaledToFit()
                    .frame(width: geo.size.height * 0.35)

                Spacer()

                BotaoOnboarding(textoBotao: "Continuar") {
                    router.push(.onboarding)
                    viewModel.continuar()
                }
                    .frame(width: geo.size.width * 0.80)
            }
            .frame(maxWidth: .infinity, maxHeight: .infinity)
            .background(Color.cordoFundo)
        }
    }
}

#Preview{
    TelaDeApresetacao(viewModel: OnboardingViewModel())
}
```

Arquivo: ./Omi/View/DetalhesReceitaView.swift

```
//      //
//      //  DetalhesReceitaView.swift
//      //  Omi
//      //
//      //  Created by Igor Carrasco on 19/08/26.
//      //
//
import SwiftUI
//import CoreData

struct DetalhesReceitaView: View {
    @Bindable var viewModel: DetalhesReceitaViewModel

    var body: some View {
        GeometryReader { geo in

            ZStack {
                Color(.cordoFundo)
                    .ignoresSafeArea()

                ScrollView {
                    VStack(alignment: .leading, spacing: 24) {

                        ZStack(alignment: .bottomLeading){
                            ReceitaHeroView(
                                imagemData: viewModel.receita?.imagem,
                                titulo: viewModel.receita?.titulo ?? ""
                            )
                            HStack(){
                                CapsulaDetalhes(detalhe:
                                    .tempo(viewModel.receita?.tempoDePreparo ?? 1))
                                    .position(
                                        x: geo.size.width * 0.15,y:
geo.size.width * 0.48)
                                CapsulaDetalhes(detalhe:
                                    .pessoas(viewModel.receita?.porcoes ?? "1"))
                                    .position(x: geo.size.width * -0.03,y:
geo.size.width * 0.48)
                            }
                        }
                    }
                    VStack(alignment: .leading, spacing: 8) {
                        TituloSecao(texto: "Descrição:")
                        CartaoClaro {
                            Text(viewModel.receita?.descricao ?? "")
                                .font(FontesApp.corpo)
                                .foregroundColor(.cordosTextos)
                        }
                    }
                }
            }
        }
    }
}
```

```

    }

    VStack(alignment: .leading, spacing: 8) {
        TituloSecao(texto: "Ingredientes:")
        CartaoClaro {
            ForEach(viewModel.receita?.ingredientes ??
[IngredienteModel(id: UUID(), nome: "None", quantidade: "0", medida:
"ml")]) { item in

                HStack(alignment: .top, spacing: 6) {
                    Text("•")
                    Text("\ (item.quantidade) \
(item.medida) de \ (item.nome)")
                }
                .font(FontesApp.corpo)
                .foregroundColor(.cordosTextos)
            }
        }
    }

    VStack(alignment: .leading, spacing: 8) {
        TituloSecao(texto: "Modo de Preparo:")
        VStack(spacing: 16) {
            ForEach(viewModel.receita?.passos ??
[PassoModel(id: UUID(), etapa: 0, nome: "None", texto: "None",
tempoEstimado: 1)]) { passo in

                ReceitaEtapaCardView(
                    numero: passo.etapa,
                    nome: passo.nome,
                    texto: passo.texto
                    //imagemData: passo.imagemPasso
                )
            }
        }
    }
    .padding(.horizontal, 20)
    .padding(.top, 12)
    .padding(.bottom, 32)
}

.toolbar {
    ToolbarItem(placement: .automatic) {
        ShareLink(
            item: "Confira a receita de \ (viewModel.receita?.titulo
?? "") no Omi!\n\n\ (viewModel.receita?.descricao ?? "")",
            subject: Text(viewModel.receita?.titulo ?? ""),
            message: Text("Enviado via Omi App")
        ) {
            Image(systemName: "square.and.arrow.up")
        }
    }

    ToolbarItem(placement: .topBarTrailing) {

```

```

        Button("Editar") {
            // Action for edit
        }
    }
}

#Preview {
    NavigationStack{
        DetalhesReceitaView(
            viewModel: DetalhesReceitaViewModel.preview
        )
    }
}

```

Arquivo: `./Omi/View/TelaInicial.swift`

```

import SwiftUI

struct TelaInicial: View {
    @Environment(AppRouter.self) private var router // necessária para
navegação
    @Bindable var viewModel: LivroReceitasViewModel
    // @Bindable var viewModelDetalhes: DetalhesReceitaViewModel
    @State private var pesquisa = ""

    var naviTitle: String {
        if !viewModel.livroAberto {
            return "Receita"
        } else {
            return "\(viewModel.categoriaAtual.rawValue)"
        }
    }

    var body: some View {

        GeometryReader { geo in

            ZStack {

                Image("fundo")
                    .resizable()
                    .ignoresSafeArea()

                VStack {

                    // Text("Receitas")
                    // .font(FontesApp.titulo)

```

```
//          .padding(.trailing, geo.size.width * 0.45)
//          .padding(geo.size.width * 0.04)

        // Livro
        LivroInterativo(
            viewModel: viewModel
        )

//          DetalhesReceitaView(viewModel: viewModelDetalhes)

// Botões de trocar página
        TrocarPagina(
            viewModel: viewModel,

            voltar: {
                withAnimation(
                    .easeInOut(duration: 0.4)
                ) {
                    viewModel.voltar()
                }
            },

            avancar: {
                if !viewModel.livroAberto {
                    return
                }

                withAnimation(
                    .easeInOut(duration: 0.4)
                ) {
                    viewModel.avancar()
                }
            }
        )
        .frame(height: geo.size.width * 0.18)
    }
}

.toolbar {

    ToolbarItemGroup(
        placement: .bottomBar
    ) {

        HStack {

            Image(
                systemName: "magnifyingglass"
            )

            TextField(
                "Pesquisar",
                text: $pesquisa
            )
        }
    }
}
```

Arquivo: ./0mi/View/ContentViewCoreDataTestes.swift

40 / 71

```

// É o contexto do Persistence, lida com a persistência no CoreData
@Environment(\.managedObjectContext) private var contexto

@State private var mostrarForm: Bool = false

var body: some View {
    NavigationStack {
        VStack{
            LivroReceitasViewSimples(viewModel:
LivroReceitasViewModel(repo: ReceitaRepositorioCoreData(context:
contexto)))
        }
        .navigationTitle("Minhas Receitas")
        .toolbar {
            ToolbarItem(placement: .primaryAction) {
                Button{
                    mostrarForm.toggle()
                } label: {
                    Image(systemName: "plus")
                }
                .sheet(isPresented: $mostrarForm) {
                    CriarReceitaView(viewModel:
CriarReceitaViewModel(repo: ReceitaRepositorioCoreData(context: contexto)))
                }
            }
        }
    }
}

#Preview {
    // recebe o .preview que é uma inicialização em ambiente controlado,
    ambiente de pre-visualização. Em ambiente de produção/buildado o banco tem
    outros elementos.
    ContentViewCoreDataTestes()
        .environment(\.managedObjectContext,
ReceitaRepositorioCoreData.preview)
}

```

Arquivo: ./Omi/View/Onboarding5.swift

```

//
//  Onboarding5.swift
//  Omi
//
//  Created by Guilherme Alves de Souza on 18/08/26.
//

import SwiftUI

```

```
struct Onboarding5: View {

    @Bindable var viewModel: OnboardingViewModel

    private var pagina: OnboardingModel {
        viewModel.paginas[4]
    }

    var body: some View {
        GeometryReader { geo in
            VStack(spacing: 0) {
                ProgressoOnboarding(
                    paginaAtual: viewModel.paginaAtual,
                    totalDePaginas: viewModel.totalDePaginas - 1
                )

                Text(pagina.titulo)
                    .font(FontesApp.titulo)
                    .foregroundColor(Color.cordosTextos)
                    .padding(.top, geo.size.height * 0.035)

                Image(pagina.imagem)
                    .resizable()
                    .scaledToFit()
                    .frame(width: geo.size.height * 0.30)

                Spacer()

                VStack{
                    Text("Ao ")
                        .font(FontesApp.tituloComTexto)
                    +
                    Text(pagina.palavraDestaque)
                        .font(FontesApp.ExtraBold)
                    +
                    Text(pagina.descricao)
                        .font(FontesApp.tituloComTexto)
                    +
                    (pagina.palavraDestaque2.map { Text($0) } ?? Text(""))
                        .font(FontesApp.ExtraBold)
                }

                .multilineTextAlignment(.center)
                .foregroundColor(Color.cordosTextos)

                Spacer()

                BotaoOnboarding(textoBotao: "Continuar") {
                    viewModel.continuar()
                    router.push(.telaInicial)
                }
                .frame(width: geo.size.width * 0.80)
            }
        }
    }
}
```

```

        .frame(maxWidth: .infinity, maxHeight: .infinity)
        .background(Color.cordoFundo)
    }
}

#Preview("Tela 5") {
    Onboarding5(
        viewModel: OnboardingViewModel()
    )
}

```

Arquivo: `./Omi/View/LivroReceitasViewSimples.swift`

```

////
//// LivroReceitasViewSimples.swift
//// Omi
////
//// Created by Igor Carrasco on 19/08/26.
////

import SwiftUI

struct LivroReceitasViewSimples: View {
    // repo j[la vem injetado por fora
    // @Bindable permite usar o $viewModel.categoriaAtual com
    Picker/conteole
    @Bindable var viewModel: LivroReceitasViewModel

    var body: some View {
        // Picker("Categoria", selection: $viewModel.categoriaAtual) {
        //     ForEach(CategoriaReceita.allCases) { categoria in
        //         Text(categoria.rawValue).tag(categoria)
        //     }
        // }
        // .pickerStyle(.segmented)
        // .onChange(of: viewModel.categoriaAtual) { _, _ in
        //     viewModel.paginaAtual = 1
        // }

        if viewModel.receitasFiltradas.isEmpty {
            ContentUnavailableView("Nenhuma receita nessa categoria",
systemImage: "book")
        } else {
            TabView(selection: $viewModel.paginaAtual) {
                ForEach(Array(viewModel.receitasFiltradas.enumerated()),
id: \.element.id) { index, receita in
                    ReceitaPageView(
                        receita: receita

```

```
        )
        .tag(index + 1) // +1 pq a página atual é 1-index
    }
}
.tabViewStyle(.page)
}
}
}

#Preview {
    LivroReceitasViewSimples(viewModel: .preview)
}
```

Arquivo: ./Omi/View/RotasDestinoView.swift

```
//
// RotasDestinoView.swift
// Omi
//
// Created by Igor Carrasco on 20/08/26.
//

import SwiftUI

// Único lugar do App que transforma Rota em View

struct RotasDestinoView: View {
    let rota: Rota

    @Environment(\.managedObjectContext) private var contexto

    var body: some View {
        switch rota {
        case .onboarding:
            OnboardingView(viewModel: OnboardingViewModel())

        case .telaInicial:
            TelaInicial(viewModel: LivroReceitasViewModel(repo:
ReceitaRepositorioCoreData(context: contexto)))

        case .detalheReceita(let receita):
            DetalhesReceitaView(viewModel:
DetalhesReceitaViewModel(receita: receita, repo:
ReceitaRepositorioCoreData(context: contexto)))

        case .criarReceita:
            CriarReceitaView(viewModel: CriarReceitaViewModel(repo:
ReceitaRepositorioCoreData(context: contexto)))
```

```
//          case .categoriaSheetView:
//              CategoriaSheetView(categoriaSelecionada: .refeicao,
//                                  aoSelecionar: { _ in } )

//          case .listaIngredientes:
//              ListaIngredientesView(viewModel: IngredientesViewModel(repo:
//                                  ReceitaRepositorioCoreData(context: contexto)))
//          }
//      }
}
```

Arquivo: `./Omi/View/CriarReceitaView.swift`

```
//
// CriarReceitaView.swift
// Omi
//
// Created by Leonardo Gonçalves da Silva on 18/08/26.
//
import SwiftUI
import PhotosUI

struct CriarReceitaView: View {
    // @Environment(AppRouter.self) private var router
    @Environment(\.dismiss) private var voltar
    @Bindable var viewModel: CriarReceitaViewModel
    @State private var itemSelecionado: PhotosPickerItem? // ✅ estado
    local do picker
    @State private var itemPassoSelecionado: PhotosPickerItem?

    var body: some View {
        NavigationStack {
            ZStack {
                Color(.cordoFundo)
                    .ignoresSafeArea()

                ScrollView {
                    VStack(alignment: .leading, spacing: 20) {

                        // MARK: Foto (agora funcional, ligada a
                        viewModel.imagem: Data?)
                        VStack(alignment: .leading, spacing: 6) {
                            Text("Adicionar foto")
                                .font(FontesApp.Semibold)
                                .foregroundColor(.cordosTextos)

                            PhotosPicker(selection: $itemSelecionado,
                                matching: .images) {
                                    HStack(spacing: 12) {
```

```

        if let dados = viewModel.imagem, let
uiImage = UIImage(data: dados) {
            Image(uiImage: uiImage)
                .resizable()
                .scaledToFill()
                .frame(width: 32, height: 32)

        .clipShape(RoundedRectangle(cornerRadius: 8))
        } else {
            Image(systemName: "photo")
                .font(.title2)
        }

        Text(viewModel.imagem == nil ?
"Selecione uma imagem" : "Imagem selecionada")
            .font(FontesApp.corpo)

        Spacer()
    }

    .foregroundColor(.cordosTextos.opacity(0.7))
    .frame(maxWidth: .infinity, alignment:
    .leading)

    .padding(.horizontal, 16)
    .padding(.vertical, 14)
    .background(Color(.corFundoCapsula))
    .clipShape(Capsule())
}
.onChange(of: itemSelecionado) { _, novoItem in
    Task {
        if let dados = try? await
novoItem?.loadTransferable(type: Data.self) {
            viewModel.imagem = dados
        }
    }
}

// Recipe Title
VStack(alignment: .leading, spacing: 6) {
    Text("Título da Receita")
        .font(FontesApp.Semibold)
        .foregroundColor(.cordosTextos)

    TextField("Ex: Bolo de chocolate", text:
$viewModel.titulo)

        .padding(.horizontal, 16)
        .padding(.vertical, 12)
        .background(Color(.corFundoCapsula))
        .clipShape(Capsule())
}

// Portions
VStack(alignment: .leading, spacing: 6) {

```

```

        Text("Porções")
            .font(FontesApp.Semibold)
            .foregroundStyle(.cordosTextos)

        TextField("Ex: 5 pessoas", text:
$viewModel.porcoesTexto)
            .padding(.horizontal, 16)
            .padding(.vertical, 12)
            .background(Color(.corFundoCapsula))
            .clipShape(Capsule())
    }

    // Prep Time
    VStack(alignment: .leading, spacing: 6) {
        Text("Tempo de preparo")
            .font(FontesApp.Semibold)
            .foregroundStyle(.cordosTextos)

        TextField("Ex: 30 min", text:
$viewModel.tempoDePreparoTexto)
            .padding(.horizontal, 16)
            .padding(.vertical, 12)
            .background(Color(.corFundoCapsula))
            .clipShape(Capsule())
    }

    // MARK: Categoria (estava faltando na sua versão –
    necessário pra salvar corretamente)
    VStack(alignment: .leading, spacing: 6) {
        Text("Categoria")
            .font(FontesApp.Semibold)
            .foregroundStyle(.cordosTextos)

        Button("BBB") {
            router.apresentarSheet(.categoriaSheetView)
        }
        Picker("Categoria", selection:
$viewModel.categoria) {
            //          ForEach(CategoriaReceita.allCases) { cat
            //              Text(cat.nomeExibicao).tag(cat)
            //          }
            //      }
            //      .pickerStyle(.menu)
            //      .padding(.horizontal, 16)
            //      .padding(.vertical, 8)
            //      .background(Color(.corFundoCapsula))
            //      .clipShape(Capsule())
        }

        // MARK: – Descrição
        VStack(alignment: .leading) {
            Text("Descrição:")

```

```

        .font(FontesApp.corpo)
        TextEditor(text: $viewModel.descricao)
        .frame(minHeight: 200)
    }

    // Ingredients Section
    VStack(alignment: .leading, spacing: 8) {
        Text("Ingredientes:")
            .font(FontesApp.Semibold)
            .foregroundColor(.cordosTextos)

        VStack(spacing: 8) {
            TextField("Ingrediente (ex: Chocolate)",
text: $viewModel.nomeIngredienteTexto)
            Divider()
            TextField("Quantidade (ex: 1)", text:
$viewModel.quantidadeTexto)
            Divider()
            TextField("Medida (ex: xícara)", text:
$viewModel.medidaTexto)

            Button(action: {
                viewModel.adicionarIngrediente()
            }) {
                Label("Adicionar Ingrediente",
systemImage: "plus")
                    .font(FontesApp.Semibold)
                    .frame(maxWidth: .infinity)
            }
                .padding(.top, 4)
        }
        .padding(16)
        .background(Color(.corFundoCapsula))
        .clipShape(RoundedRectangle(cornerRadius: 20))

        ForEach(viewModel.ingredientesAdicionados) {
item in
            Text("• \$(item.quantidade.formatted()) \
(item.medida) - \$(item.nome)")
                .font(FontesApp.corpo)
                .foregroundColor(.cordosTextos)
                .padding(.leading, 8)
        }
    }

    // MARK: - Modo de preparo Section
    VStack(alignment: .leading, spacing: 8) {
        Text("Modo de Preparo:")
            .font(FontesApp.Semibold)
            .foregroundColor(.cordosTextos)

        VStack(alignment: .leading, spacing: 8) {

```

```

        TextField("Nome da etapa", text:
$viewModel.nomeDoPasso)

        Divider()
        Text("Descrição do Passo")
            .font(FontesApp.corpo)
        TextEditor(text:
$viewModel.descricaoDoPasso)
            .frame(minHeight: 80)
            .scrollContentBackground(.hidden)
            .background(Color.white.opacity(0.5))

        .clipShape(RoundedRectangle(cornerRadius: 10))

        Button(action: {
            viewModel.adicionarPasso()
        }) {
            Label("Adicionar Etapa", systemImage:
"plus")
                .font(FontesApp.Semibold)
                .frame(maxWidth: .infinity)
        }
    }
    .padding(16)
    .background(Color(.corFundoCapsula))
    .clipShape(RoundedRectangle(cornerRadius: 20))

    ForEach(viewModel.passosAdicionados) { passo in
        Text("Etapa \(${passo.etapa}): \(${passo.nome}")
            .font(FontesApp.corpo)
            .foregroundStyle(.cordosTextos)
            .padding(.leading, 8)
        }
    }
    .padding(20)
}

Receita")
    .navigationTitle(viewModel.modos ? "Editar Receita" : "Anotar
Receita")
    .navigationBarTitleDisplayMode(.inline)
    .toolbar {
        ToolbarItem(placement: .topBarLeading) {
            Button(action: { voltar() }) {
                Image(systemName: "xmark")
            }
        }

        ToolbarItem(placement: .topBarTrailing) {
            Button(action: {
                viewModel.salvarReceitaNoBanco()
                voltar()
            }) {
                Image(systemName: "checkmark")
            }
        }
    }
}

```

```

        }
        .buttonStyle(.borderedProminent)
        .buttonBorderShape(.circle)
        .tint(.green)
    }
}
}
}
}
//
// #Preview("Criar") {
//     CriarReceitaView(viewModel: .preview)
// }

#Preview {
    CriarReceitaView(viewModel: .preview)
        .environment(AppRouter())
}

```

Arquivo: `./Omi/View/Onboarding3.swift`

```

//
// Onboarding3.swift
// Omi
//
// Created by Guilherme Alves de Souza on 18/08/26.
//

import SwiftUI

struct Onboarding3: View {

    @Bindable var viewModel: OnboardingViewModel

    private var pagina: OnboardingModel {
        viewModel.paginas[2]
    }

    var body: some View {
        GeometryReader { geo in
            VStack(spacing: 0) {
                ProgressoOnboarding(
                    paginaAtual: viewModel.paginaAtual,
                    totalDePaginas: viewModel.totalDePaginas - 1
                )
            }
        }
    }
}

```

```
        Text(pagina.titulo)
            .font(FontesApp.titulo)
            .foregroundColor(.black.opacity(0.7))
            .padding(.top, geo.size.height * 0.035)

        Spacer()

        Image(pagina.imagem)
            .resizable()
            .scaledToFit()
            .frame(width: geo.size.height * 0.25)

        Spacer()

        VStack{
            Text(pagina.descricao)
                .font(FontesApp.tituloComTexto)
            +
            Text(pagina.palavraDestaque)
                .font(FontesApp.ExtraBold)
            +
            (pagina.palavraDestaque2.map { Text($0) } ?? Text(""))
                .font(FontesApp.tituloComTexto)
        }
        .multilineTextAlignment(.center)
        .foregroundColor(.black.opacity(0.7))

        Spacer()

        BotaoOnboarding(textoBotao: "Continuar") {
            viewModel.continuar()
        }
        .frame(width: geo.size.width * 0.80)
    }
    .frame(maxWidth: .infinity, maxHeight: .infinity)
    .background(Color.cordoFundo)
}

}

}

#Preview("Tela 3") {
    Onboarding3(
        viewModel: OnboardingViewModel()
    )
}
```

```
//  
// Onboarding1.swift  
// On bording  
//  
// Created by Guilherme Alves de Souza on 16/08/26.  
//  
  
import SwiftUI  
  
struct Onboarding1: View {  
  
    @Bindable var viewModel: OnboardingViewModel  
  
    private var pagina: OnboardingModel {  
        viewModel.paginas[0]  
    }  
  
    var body: some View {  
        GeometryReader { geo in  
            VStack{  
                ProgressoOnboarding(  
                    paginaAtual: viewModel.paginaAtual,  
                    totalDePaginas: viewModel.totalDePaginas - 1  
                )  
  
                Text(pagina.titulo)  
                    .font(FontesApp.titulo)  
                    .foregroundColor(Color.cordosTextos)  
                    .padding(.top, geo.size.height * 0.035)  
  
                Spacer()  
  
                Image(pagina.imagem)  
                    .resizable()  
                    .scaledToFit()  
                    .frame(width: geo.size.height * 0.35)  
  
                Spacer()  
  
                VStack{  
                    Text(pagina.descricao)  
                        .font(FontesApp.tituloComTexto)  
                    +  
                    Text(pagina.palavraDestaque)  
                        .font(FontesApp.ExtraBold)  
                }  
  
                .multilineTextAlignment(.center)  
                .foregroundColor(Color.cordosTextos)  
            }  
        }  
    }  
}
```

```
        Spacer()

        BotaoOnboarding(textoBotao: "Continuar") {
            viewModel.continuar()
        }
        .frame(width: geo.size.width * 0.80)
    }
    .frame(maxWidth: .infinity, maxHeight: .infinity)
    .background(Color.cordoFundo)
}
}

#Preview("Tela 1") {
    Onboarding1(
        viewModel: OnboardingViewModel()
    )
}
```

Arquivo: ./Omi/View/Onboarding4.swift

```
//
//  Onboarding4.swift
//  Omi
//
//  Created by Guilherme Alves de Souza on 18/08/26.
//

import SwiftUI

struct Onboarding4: View {

    @Bindable var viewModel: OnboardingViewModel

    private var pagina: OnboardingModel {
        viewModel.paginas[3]
    }

    var body: some View {
        GeometryReader { geo in
            VStack(spacing: 0) {
                ProgressoOnboarding(
                    paginaAtual: viewModel.paginaAtual,
                    totalDePaginas: viewModel.totalDePaginas - 1
                )

                Text(pagina.titulo)
                    .font(FontesApp.titulo)
            }
        }
    }
}
```

```
        .foregroundColor(.black.opacity(0.7))
        .padding(.top, geo.size.height * 0.035)

    Spacer()

    Image(pagina.imagem)
        .resizable()
        .scaledToFit()
        .frame(width: geo.size.height * 0.25)

    Spacer()

    VStack{
        Text(pagina.descricao)
            .font(FontesApp.tituloComTexto)
        +
        Text(pagina.palavraDestaque)
            .font(FontesApp.ExtraBold)
        +
        (pagina.palavraDestaque2.map { Text($0) } ?? Text(""))
            .font(FontesApp.tituloComTexto)
    }
        .multilineTextAlignment(.center)
        .foregroundColor(.black.opacity(0.7))

    Spacer()

    BotaoOnboarding(textoBotao: "Continuar") {
        viewModel.continuar()
    }
        .frame(width: geo.size.width * 0.80)
    }
    .frame(maxWidth: .infinity, maxHeight: .infinity)
    .background(Color.cordoFundo)
}
}

#Preview("Tela 4") {
    Onboarding4(
        viewModel: OnboardingViewModel()
    )
}
```

Arquivo: ./Omi/View/OnboardingView.swift

```
//  
// OnboardingView.swift  
// On bording  
//  
// Created by Guilherme Alves de Souza on 17/08/26.  
//  
  
import SwiftUI  
  
struct OnboardingView: View {  
    @Environment(AppRouter.self) private var router  
    @Bindable var viewModel = OnboardingViewModel()  
  
    var body: some View {  
        Group{  
            switch viewModel.paginaAtual {  
            case 0:  
                TelaDeApresetacao(viewModel: viewModel)  
            case 1:  
                Onboarding1(viewModel: viewModel)  
            case 2:  
                Onboarding2(viewModel: viewModel)  
            case 3:  
                Onboarding3(viewModel: viewModel)  
            case 4:  
                Onboarding4(viewModel: viewModel)  
            // case 5:  
            default:  
                Onboarding5(viewModel: viewModel)  
            // default:  
            //     EmptyView()  
            }  
        }  
        .onChange(of: viewModel.finalizado) { _, finalizado in  
            if finalizado {  
                router.finalizarOnboarding()  
            }  
        }  
    }  
}  
  
#Preview {  
    OnboardingView()  
        .environment(AppRouter())  
}
```

Arquivo: ./Omi/View/Onboarding2.swift

```
//  
// Onboarding2.swift  
// On bording  
//  
// Created by Guilherme Alves de Souza on 17/08/26.  
//  
  
import SwiftUI  
struct Onboarding2: View {  
  
    @Bindable var viewModel: OnboardingViewModel  
  
    private var pagina: OnboardingModel {  
        viewModel.paginas[1]  
    }  
  
    private let postits = [  
        "Etiqueta laranja",  
        "Etiqueta amarela",  
        "Etiqueta verde",  
        "Etiqueta azul",  
        "Etiqueta vermelha"  
    ]  
  
    var body: some View {  
        GeometryReader { geometry in  
  
            VStack() {  
  
                ProgressoOnboarding(  
                    paginaAtual: viewModel.paginaAtual,  
                    totalDePaginas: viewModel.totalDePaginas - 1  
                )  
  
                Text(pagina.titulo)  
                    .font(FontesApp.titulo)  
                    .foregroundColor(Color.cordosTextos)  
                    .padding(.top, geometry.size.height * 0.035)  
  
                Spacer()  
  
                ZStack() {  
  
                    HStack(spacing: 2) {  
                        ForEach(postits.indices, id: \.self) { index in  
  
                            PostitOnboarding(imagem: postits[index],ativo:  
viewModel.postitAtivo == index)  
                                .frame(width: geometry.size.width * 0.099)  
                        }  
                    }  
                }  
            }  
        }  
    }  
}
```

```
.padding(.leading, geometry.size.height * 0.06)
.offset(y: -geometry.size.height * 0.23)
Image(pagina.imagem)
    .resizable()
    .scaledToFit()
    .frame(width: geometry.size.width * 0.70)
}

VStack{
    Text(pagina.descricao)
    +
    Text(pagina.palavraDestaque)
        .font(FontesApp.ExtraBold)
    Text("com ")
    +
    (pagina.palavraDestaque2.map { Text($0) } ?? Text(""))
        .font(FontesApp.ExtraBold)
}

.font(FontesApp.tituloComTexto)
.multilineTextAlignment(.center)
.foregroundColor(Color.cordosTextos)

Spacer()

BotaoOnboarding(textoBotao: "Continuar") {
    viewModel.continuar()
}
.frame(width: min(geometry.size.width * 0.78, 305))
}
.frame(width: geometry.size.width, height: geometry.size.height)
.background(Color.cordoFundo)
}
.task {
    await viewModel.iniciarAnimacaoPostIts()
}
}

#Preview("Tela 2") {
    Onboarding2(
        viewModel: OnboardingViewModel()
    )
}
```

Arquivo: ./Omi/OmiApp.swift

```
//  
// OmiApp.swift  
// Omi  
//  
// Created by Leonardo Gonçalves da Silva on 14/08/26.  
//  
  
import SwiftUI  
import CoreData  
  
@main  
struct OmiApp: App {  
    let persistentController = PersistenceController.shared  
    @State private var router = AppRouter()  
  
    // Le do UserDefaults quando é finalizado pelo finalizarOnboarding()  
    // @AppStorage observa a notificação de mudança  
    @AppStorage("onboardingConcluido") private var onboardingConcluido:  
    Bool = false  
  
    var body: some Scene {  
        WindowGroup {  
            NavigationStack(path: $router.path) {  
                Group{  
                    if onboardingConcluido {  
                        TelaInicial(viewModel: LivroReceitasViewModel(repo:  
ReceitaRepositorioCoreData(context:  
persistentController.container.viewContext)))  
                    } else {  
                        OnboardingView()  
                    }  
                }  
                .navigationDestination(for: Rota.self) { rota in  
                    RotasDestinoView(rota: rota)  
                }  
            }  
            .environment(\.managedObjectContext,  
persistentController.container.viewContext)  
            .environment(router)  
            // Apresenta o criatReceita como um modal, separada da pilha  
(path). Usa o mesmo RotasDestinoView  
            // Muda só como a tela aparece (como sheet, não empurrada na  
pilha).  
            .sheet(item: $router.sheetAtual) { rota in  
                RotasDestinoView(rota: rota)  
                .environment(\.managedObjectContext,  
persistentController.container.viewContext) // Passa o environment  
novamente pra evitar perda de contexto no futuro, explicitando qual é o  
contexto a ser observado  
            }  
        }  
    }  
}
```

```
}
```

Arquivo: `./Omi/Font/Fontes.swift`

```
//  
// S.swift  
// Omi  
//  
// Created by Guilherme Alves de Souza on 17/08/26.  
//  
  
import SwiftUI  
  
enum FontesApp {  
  
    static let titulo = Font.custom(  
        "Dosis",  
        size: 32,  
        relativeTo: .largeTitle  
    )  
    .weight(.bold)  
  
    static let tituloComTexto = Font.custom(  
        "Dosis",  
        size: 32,  
        relativeTo: .largeTitle  
    )  
    .weight(.semibold)  
  
    static let subtitulo = Font.custom(  
        "Dosis",  
        size: 20,  
        relativeTo: .title3  
    )  
    .weight(.semibold)  
  
    static let corpo = Font.custom(  
        "Dosis",  
        size: 17,  
        relativeTo: .body  
    )  
    .weight(.medium)  
  
    static let ExtraBold = Font.custom(  
        "Dosis",  
        size: 32,  
        relativeTo: .body
```

```
)  
    .weight(.heavy)  
  
static let Botao = Font.custom(  
    "Dosis",  
    size: 16,  
    relativeTo: .body  
)  
    .weight(.heavy)  
  
static let Semibold = Font.custom(  
    "Dosis",  
    size: 14,  
    relativeTo: .body  
)  
    .weight(.semibold)  
}
```

Arquivo: `./Omi/Data/PreviewData.swift`

```
//  
// PreviewData.swift  
// Omi  
//  
// Created by Igor Carrasco on 19/08/26.  
//  
  
import Foundation  
import CoreData  
  
enum PreviewData {  
    static func receitas(context: NSManagedObjectContext) {  
        for i in 1...5 {  
            let receita = Receita(context: context)  
  
            receita.id = UUID()  
            receita.titulo = "Receita \(i)"  
        }  
  
        try? context.save()  
    }  
}  
  
// Como aqui é sobre Preview e também importa o CoreData, posso chamar  
// extensions do que preciso criar para passar chamar no preview  
extension LivroReceitasViewModel {
```

```

        static var preview: LivroReceitasViewModel {
            LivroReceitasViewModel(repo: ReceitaRepositorioCoreData(context:
PersistenceController.preview.container.viewContext))
        }
    }

// .preview para o ContentViewCoreDataTestes
extension ReceitaRepositorioCoreData {
    static var preview: NSManagedObjectContext {
        PersistenceController.preview.container.viewContext
    }
}

extension CriarReceitaViewModel {
    static var preview: CriarReceitaViewModel {
        CriarReceitaViewModel(repo: ReceitaRepositorioCoreData(context:
PersistenceController.preview.container.viewContext))
    }
}

extension DetalhesReceitaViewModel {
    static var preview: DetalhesReceitaViewModel {
        DetalhesReceitaViewModel(
            receita: ReceitaModel(
                id: UUID(),
                titulo: "Bolo de Cenoura",
                categoria: .sobremesa,
                descricao: "O bolo de cenoura é uma receita clássica e
amada por todos! Com sua massa fofinha e saborosa, esse bolo é perfeito
para um lanche da tarde ou como sobremesa em qualquer ocasião.",
                imagem: nil,
                tempoDePreparo: 40,
                porcoes: "8",
                dificuldade: nil,
                dataCriacao: Date(),
                dataAtualizacao: nil,
                ingredientes: [
                    IngredienteModel(id: UUID(), nome: "cenouras médias",
quantidade: "3", medida: "unidades"),
                    IngredienteModel(id: UUID(), nome: "ovos", quantidade:
"4", medida: "unidades")
                ],
                passos: [
                    PassoModel(id: UUID(), etapa: 1, nome: "Massa do Bolo",
texto: "Bata as cenouras, os ovos e o óleo no liquidificador.",
tempoEstimado: 40),
                    PassoModel(id: UUID(), etapa: 2, nome: "Massa do Bolo",
texto: "Bata as cenouras, os ovos e o óleo no liquidificador.",
tempoEstimado: 40)
                ]
            ),
        )
    }
}

```

```
        repo: ReceitaRepositorioCoreData(context:
PersistenceController.preview.container.viewContext)
    )
}
}
```

Arquivo: ./Omi/Data/ReceitaRepositorioCoreData.swift

```
//
//  ReceitasRepo.swift
//  Omi
//
//  Created by Leonardo Gonçalves da Silva on 17/08/26.
//
import CoreData

// Implementação concreta usando CoreData
final class ReceitaRepositorioCoreData: ReceitaRepositorio {
    private let contexto: NSManagedObjectContext

    init(context: NSManagedObjectContext) {
        self.contexto = context
    }

    private func saveData() {
        do {
            try contexto.save()
        } catch {
            contexto.rollback()
            print("ERRO AO SALVAR COREDATA: \(error.localizedDescription)")
        }
    }

    // MARK: - Create

    // O segredo para não duplicar dados!
    private func buscarOuCriarIngrediente(nome: String) throws ->
    Ingrediente {
        let nomeLimpo = nome.trimmingCharacters(in: .whitespaces)

        let request = NSFetchRequest<Ingrediente>(entityName:
        "Ingrediente")
        // O '[c]' no final do %m avisa o Core Data para ignorar maiúsculas
        e minúsculas!
        // Assim "Chocolate" e "chocolate" são reconhecidos como o mesmo
        item.
        request.predicate = NSPredicate(format: "nome == [c] %@",
        nomeLimpo)
        request.fetchLimit = 1
    }
}
```

```
    let resultados = try contexto.fetch(request)

    // Se já existe, retorna o que achou no banco
    if let ingredienteExistente = resultados.first {
        return ingredienteExistente
    } else {
        // Se não existe, cria um novo (mas não dá save ainda, vamos
salvar tudo junto com a receita)
        let novoIngrediente = Ingrediente(context: contexto)
        novoIngrediente.id = UUID()
        novoIngrediente.nome = nomeLimpo
        return novoIngrediente
    }
}

func criarReceita(
    titulo: String,
    categoria: String,
    descricao: String,
    imagem: Data?,
    tempoDePreparo: Int16,
    porcoes: String,
    dificuldade: String?,
    ingredientes: [IngredienteAdicionado],
    passos: [PassoAdicionado]
) throws {

    // 1. Cria a Receita principal
    let receita = Receita(context: contexto)

    receita.id = UUID()
    receita.titulo = titulo
    receita.categoria = categoria
    receita.descricao = descricao
    receita.imagem = imagem
    receita.tempoDePreparo = tempoDePreparo
    receita.porcoes = porcoes
    receita.dificuldade = dificuldade
    receita.dataCriacao = Date()

    // 2. Loop passando por todos os ingredientes que o usuário
escolheu
    for item in ingredientes {
        let ingrediente = try buscarOuCriarIngrediente(nome: item.nome)
        // Cria a entidade intermediária
        let relacao = IngredienteDaReceita(context: contexto)

        relacao.id = UUID()
        relacao.quantidade = String(item.quantidade)
        relacao.medida = item.medida
        relacao.receita = receita
        relacao.ingrediente = ingrediente
    }
}
```

```
// CRIAR OS PASSOS
for item in passos {
    let passo = PassoReceita(context: contexto)

    passo.id = UUID()
    passo.etapa = Int16(item.etapa)
    passo.nome = item.nome
    passo.texto = item.texto
    passo.imagem = imagem
    passo.tempoEstimado = Int16(item.tempoEstimado)
    passo.receita = receita
}

// 4. Salva o contexto (Isso salva a Receita e todas as relações
ReceitaIngrediente de uma vez só)
//      try context.save()
saveData()
}

// MARK: - Read
// Como devolve um [ReceitaModel] e não um [Receita], ele fecha o
"vazamento"

func buscarReceitas() throws -> [ReceitaModel] {
    let request = NSFetchRequest<Receita>(entityName: "Receita")
    request.sortDescriptors = [NSSortDescriptor(keyPath:
\Receita.dataCriacao, ascending: false)]
    let entities = try contexto.fetch(request)
    return entities.map { $0.toModel() }
}

func buscarReceita(id: UUID) throws -> ReceitaModel? {
    try buscarReceitaEntity(id: id)?.toModel()
}

func buscarIngredientes() throws -> [IngredienteCadastradoModel] {
    let request = NSFetchRequest<Ingrediente>(entityName:
"Ingrediente")
    request.sortDescriptors = [NSSortDescriptor(keyPath:
\Ingrediente.nome, ascending: true)]
    let entities = try contexto.fetch(request)
    return entities.map { $0.toModel() }
}

func buscarPassos(para receita: Receita) throws -> [PassoReceita] {
    let request = NSFetchRequest<PassoReceita>(entityName:
"PassoReceita")

    // Predicate: "Only give me the steps where the recipe matches the
one I passed in"
    request.predicate = NSPredicate(format: "receita == %@", receita)

    // Sort: Put them in ascending order based on the 'etapa' number
```

```
        request.sortDescriptors = [NSSortDescriptor(keyPath:
\PassoReceita.etapa, ascending: true)]

        return try contexto.fetch(request)
    }

    // MARK: – Helpers privados de busca por id
    // Um dos poucos lugares do app que ainda "pensa"em NSManagedObject
    private func buscarReceitaEntity(id: UUID) throws -> Receita? {
        let request = NSFetchRequest<Receita>(entityName: "Receita")
        request.predicate = NSPredicate(format: "id == %@", id as CVarArg)
        request.fetchLimit = 1
        return try contexto.fetch(request).first
    }

    private func buscarPassoEntity(id: UUID) throws -> PassoReceita? {
        let request = NSFetchRequest<PassoReceita>(entityName:
"PassoReceita")
        request.predicate = NSPredicate(format: "id == %@", id as CVarArg)
        request.fetchLimit = 1
        return try contexto.fetch(request).first
    }

    private func buscarIngredienteEntity(id: UUID) throws -> Ingrediente? {
        let request = NSFetchRequest<Ingrediente>(entityName:
"Ingrediente")
        request.predicate = NSPredicate(format: "id == %@", id as CVarArg)
        request.fetchLimit = 1
        return try contexto.fetch(request).first
    }

    private func buscarRelacaoEntity(id: UUID) throws ->
IngredienteDaReceita? {
        let request = NSFetchRequest<IngredienteDaReceita>(entityName:
"IngredienteDaReceita")
        request.predicate = NSPredicate(format: "id == %@", id as CVarArg)
        request.fetchLimit = 1
        return try contexto.fetch(request).first
    }

    // MARK: – Update

    func atualizarReceita(
        id: UUID,
        novoTitulo: String,
        novaCategoria: String,
        novaDescricao: String,
        novaImagem: Data?,
        novoTempoDePreparo: Int16,
        novasPorcoes: String,
        novaDificuldade: String?
    ) throws {
        guard let receita = try buscarReceitaEntity(id: id) else { return }
    }
```

```
        receita.titulo = novoTitulo
        receita.categoria = novaCategoria
        receita.descricao = novaDescricao
        receita.imagem = novaImagem
        receita.tempoDePreparo = novoTempoDePreparo
        receita.porcoes = novasPorcoes
        receita.dificuldade = novaDificuldade
        receita.dataAtualizacao = Date()

        //          try context.save()
        saveData()
    }

    func atualizarPasso(
        id: UUID,
        novaEtapa: Int16,
        novoNome: String,
        novoTexto: String,
        novaImagem: Data?,
        novoTempoEstimado: Int16

    )throws{
        guard let passo = try buscarPassoEntity(id: id) else { return }

        passo.etapa = novaEtapa
        passo.nome = novoNome
        passo.texto = novoTexto
        passo.imagem = novaImagem
        passo.tempoEstimado = novoTempoEstimado

        //          try context.save()
        saveData()
    }

    func atualizarIngredienteDaReceita(
        id: UUID,
        novaQuantidade: String,
        novaMedida: String
    )throws{
        guard let relacao = try buscarRelacaoEntity(id: id) else { return }

        relacao.quantidade = novaQuantidade
        relacao.medida = novaMedida

        //          try context.save()
        saveData(    )
    }

    // MARK: – Criar passo avulso (fora da criação de receita completa)
    func criarPasso(
        receitaId: UUID,
        etapa: Int16,
        nome: String,
        texto: String,
```

```
        imagem: Data?,
        tempoEstimado: Int16
    ) throws {
        guard let receita = try buscarReceitaEntity(id: receitaId) else {
return }

        let passo = PassoReceita(context: contexto)
        passo.id = UUID()
        passo.etapa = etapa
        passo.nome = nome
        passo.texto = texto
        passo.imagem = imagem
        passo.tempoEstimado = tempoEstimado
        passo.receita = receita

        saveData()
    }

    func criarIngredienteAvulso(nome: String) throws ->
IngredienteCadastradoModel {
        let ingrediente = try buscarOuCriarIngrediente(nome: nome)
        saveData()
        return ingrediente.toModel()
    }

    // MARK: - Delete
    // Busca entity pelo id e deleta - quem chama não precisa ter uma
referência do NSManagedObject
    func deletarReceita(id: UUID) throws {
        guard let receita = try buscarReceitaEntity(id: id) else { return }

        contexto.delete(receita)
        saveData()
    }

    func deletarPasso(id: UUID) throws {
        guard let passo = try buscarPassoEntity(id: id) else { return }

        contexto.delete(passo)
        saveData()
    }

    func deletarIngrediente(id: UUID) throws {
        guard let ingrediente = try buscarIngredienteEntity(id: id) else {
return }

        contexto.delete(ingrediente)
        saveData()
    }
}
```

Arquivo: ./Omi/Data/Persistence.swift

```
//
// Persistence.swift
// Omi
//
// Created by Leonardo Gonçalves da Silva on 14/08/26.
//

import CoreData

// O NSPersistentContainer monta o NSManagedObjectModel, o
// NSPersistentStoreCoordinator e entrega um NSManagedObjectContext
// assim o container guarda contexto (rascunho)
struct PersistenceController {
    static let shared = PersistenceController() // É um container que cuida
do viewContext do CoreData

    // Declara-se o container e precisa ter o mesmo nome do
arquivo.xdatamodeld
    let container: NSPersistentContainer

    // Inicializa o container e lida com possível erro
    init(emMemoria: Bool = false) {
        container = NSPersistentContainer(name: "Omi")

        if emMemoria {
            container.persistentStoreDescriptions.first?.url =
URL(fileURLWithPath: "dev/null")
        }

        container.loadPersistentStores { descricao, erro in
            if let erro = erro {
                fatalError("ERRO AO CARREGAR CORE DATA: \
(erro.localizedDescription)")
            }
        }

        // Configura o container para mesclar automaticamente as mudanças
vindas de outros contextos
        container.viewContext.automaticallyMergesChangesFromParent = true

        container.viewContext.mergePolicy =
NSMergeByPropertyObjectTrumpMergePolicy
    }

    // Para aplicar nas #Previews das views filhas do App.swift
    @MainActor
    static let preview: PersistenceController = {

        let controller = PersistenceController()
```

```

        let context = controller.container.viewContext

//        let categorias = ["sobremesa", "salgado", "bebida", "massa",
"lanche"]
//
//        for i in 1...5 {
//            let receita = Receita(context: context)
//
//            receita.id = UUID()
//            receita.titulo = "Receita \(i)"
//            receita.descricao = "Descrição da receita: \(i)"
//            receita.categoria = categorias.randomElement()
//            receita.tempoDePreparo = Int16(20 + i)
//            receita.porcoes = "8"
//            receita.dataCriacao = Date()
//        }
//
//        try? context.save()

        return controller
    }()
}

```

Arquivo: [./Omi/Data/ReceitaRepositorioProtocolo.swift](#)

```

//
//  ReceitasRepositorio.swift
//  Omi
//
//  Created by Igor Carrasco on 19/08/26.
//

import Foundation

// Contrato de acesso a dados, facilita a migração. Quando trocar para
SwiftData, basta criar um ReceitaRepositorioSwiftData( ReceitaRepositorio)
protocol ReceitaRepositorio {
    // Receita
    func buscarReceitas() throws -> [ReceitaModel] // Busca todo o banco de
dados e retorna um ARRAY de receitas
    func buscarReceita(id: UUID) throws -> ReceitaModel? // Busca direto
pelo id. Mais eficiente e retornar UMA receita específica se encontrar

    func criarReceita(
        titulo: String,
        categoria: String,
        descricao: String,

```

```
        imagem: Data?,
        tempoDePreparo: Int16,
        porcoes: String,
        dificuldade: String?,
        ingredientes: [IngredienteAdicionado],
        passos: [PassoAdicionado]
    ) throws

func atualizarReceita(
    id: UUID,
    novoTitulo: String,
    novaCategoria: String,
    novaDescricao: String,
    novaImagem: Data?,
    novoTempoDePreparo: Int16,
    novasPorcoes: String,
    novaDificuldade: String?
) throws

func deletarReceita(id: UUID) throws

// Passo

func buscarPassos(para receita: Receita) throws -> [PassoReceita]

func criarPasso(
    receitaId: UUID,
    etapa: Int16,
    nome: String,
    texto: String,
    imagem: Data?,
    tempoEstimado: Int16
) throws

func atualizarPasso(
    id: UUID,
    novaEtapa: Int16,
    novoNome: String,
    novoTexto: String,
    novaImagem: Data?,
    novoTempoEstimado: Int16
) throws

func deletarPasso(id: UUID) throws

// Ingredientes
func buscarIngredientes() throws -> [IngredienteCadastradoModel]

func deletarIngrediente(id: UUID) throws

func atualizarIngredienteDaReceita(
    id: UUID,
    novaQuantidade: String,
    novaMedida: String
```

```
    ) throws  
  
    func criarIngredienteAvulso(nome: String) throws ->  
    IngredienteCadastradoModel  
}
```
